CSE 421

Lab 2: Introduction to Packet Analysis and Network Simulation

ID: _____

Introduction:

In real life, network monitoring and packet analysis is of great importance to observe the network state and identify any anomalous network event. Wireshark is a tool that can be used to monitor our network and also analysis traces of past network events. In this lab, we will get familiar with the wireshark interface and learn basic packet analysis.

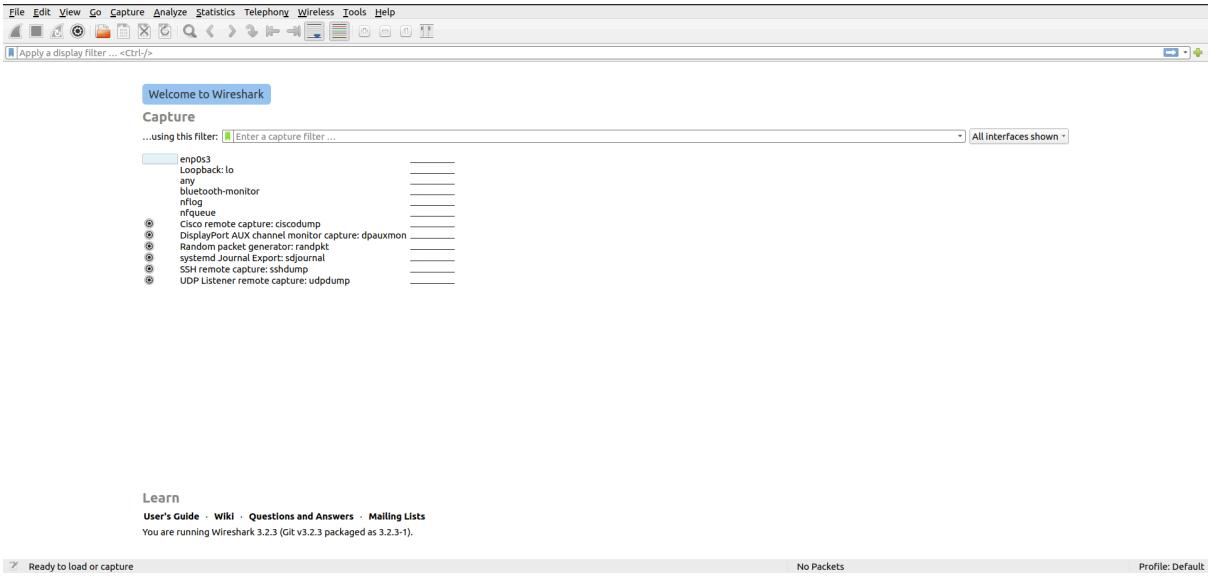
In networking research and protocol testing, simulating a network is one of the core works. We can try out existing networking protocols and measure their performance. Also, we can devise our own modified protocol and test its performance against the existing ones. In this lab, we will also get familiar with an open source network simulator, ns3. We will learn how to install the simulator and run a basic simulation.

Objectives:

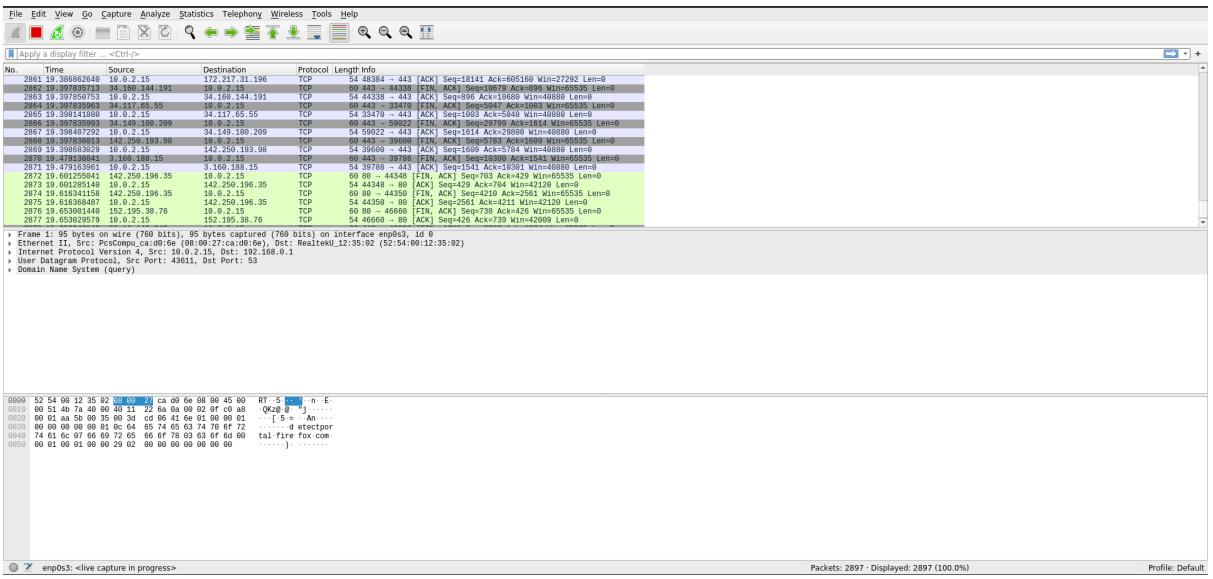
1. Getting familiar with wireshark
    a. Learn about basic wireshark interface
    b. Basic filtering of packets
    c. Explore a simple **HTTP** stream
2. Getting familiar with network simulation
    a. Installing and running ns3
    b. Understanding a simple simulation script
    c. Visualizing our simulation
    d. Collecting network traces and observing the events

Task 1: Using Wireshark [Do this task in Windows]

1. Step 1: Download and install wireshark
    a. Go to https://www.wireshark.org/
    b. Download and install the tool for the operating system you are using
2. Open wireshark if it is installed
    a. You will see a window like the following



b. You will see a list of network interfaces in your device. We will select our ethernet interface/ wifi interface, whatever we are using. In our case, we will select **enp0s3** and start capturing. To start capturing, we can double click the intended interface on which we want to capture the packets. We will see an interface like the following:



Here, there are three sub windows. The top sub window shows us the list of packets. The middle sub window shows us the headers and data of each layer of any selected packet. The bottom sub window shows us the raw bytes that each packet contains and also the decoded form of it.

c. We can observe that there are a lot of things going on. That means, our device is interacting through the network interface. We can focus on things by filtering out packets that are of our interest. For example, if we want to filter out http data, we can type ‘http’ on the text box on top and press enter. Only the http packets will be shown and other packets will be filtered out.

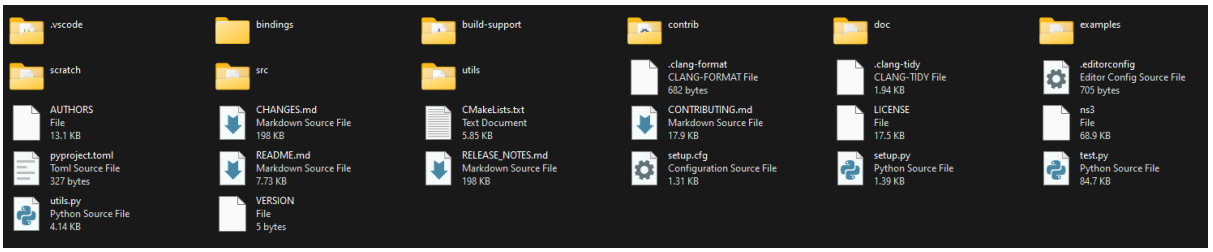
Home Task:

Visit any http website while monitoring the network traffic. Now filter only the http packets. Observe the headers of any http request and response packet and explain how the layers are operating in this case.

Task 2: Using NS-3 [Do this task in Ubuntu]

1. Download NS3
    a. Go to https://www.nsnam.org/releases/ns-3-40/ and download the exact version of ns3
        i. If the above link does not work, try from here
    b. Extract the contents and you will see the contents like this:

    c. Move inside ns-3.40 directory. The contents will look like this:



2. Install the necessary packages
    a. sudo apt install g++ python3 cmake ninja-build git
        i. If you find any error giving this command, update and upgrade your apps first and then try the above command. To update and upgrade give command,
            - **sudo apt update**
            - **sudo apt upgrade**
            - **sudo apt install g++ python3 cmake ninja-build git**
    b. python3 -m pip install --user cppyy==2.4.1
        i. If you see that pip is not installed, first install pip and then try the above command. To install pip give command,
            - **sudo apt install python3-pip**
            - **python3 -m pip install --user cppyy==2.4.1**
3. Configure ns3 with python support
    a. First go to ns-allinone-3.40/ns-3.40 directory in terminal and then command,
        i. ./ns3 configure --enable-python-bindings
4. Build ns3
    a. ./ns3 build
5. Run your first script
    a. ./ns3 shell
    b. python3 examples/tutorial/first.py
6. Try to understand our first simulation script
    a. Create nodes/devices and connect them
        i. Creating nodes:

```
nodes = ns.network.NodeContainer()
nodes.Create(2)
```

We create a NodeContainer and create multiple nodes inside that NodeContainer so that we can manage all the nodes at once through the NodeContainer

        ii. Create channel and set attributes:

```
pointToPoint = ns.point_to_point.PointToPointHelper()
pointToPoint.SetDeviceAttribute("DataRate", ns.core.StringValue("5Mbps"))
pointToPoint.SetChannelAttribute("Delay", ns.core.StringValue("2ms"))
```

We need to create a channel that will be used for the communication. In this case, we are using a point to point channel by getting a PointToPointHelper object. This PointToPointHelper will manage all the low level transmission related works.
Then we set some attributes of this channel. Here we are setting the DataRate of our channel as 5Mbps and the delay of the channel as 2ms.

iii. Connect the created nodes using the point-to-point channel:

```
devices = pointToPoint.Install(nodes)
```

Installing the channel to the nodes gives us the NetDevice that is used for network communication.

b. Install the internet stack on the nodes:

```
stack = ns.internet.InternetStackHelper()
stack.Install(nodes)
```

We need to install the network stack on the nodes so that the nodes follow the common network layering and protocols. The InternetStackHelper does the necessary works to install the network stack on the nodes.

c. Assign IP addresses to devices:

```
address = ns.internet.Ipv4AddressHelper()
address.SetBase(ns.network.Ipv4Address("10.1.1.0"),
                ns.network.Ipv4Mask("255.255.255.0"))

interfaces = address.Assign(devices)
```

We need to uniquely identify the devices in the network for communication. For this reason, we need to assign IP addresses to each devices. So, we first create a network with a network address and subnet mask. Then, we assign IP addresses to each devices. The Ipv4AddressHelper does all the necessary works to do that.

d. Creating server and client application:

i. Configuring the server:

```
echoServer = ns.applications.UdpEchoServerHelper(9)

serverApps = echoServer.Install(nodes.Get(1))
serverApps.Start(ns.core.Seconds(1.0))
serverApps.Stop(ns.core.Seconds(10.0))
```

We first create a server application that listens on its port number 9 and accepts connections from any host. Then we install the server application on the second node (nodes.Get(1), indexing starts from 0). Then we define when the server application will start and when it will stop.

ii. Configuring the client:

```
address = interfaces.GetAddress(1).ConvertTo()
echoClient = ns.applications.UdpEchoClientHelper(address, 9)
echoClient.SetAttribute("MaxPackets", ns.core.UintegerValue(1))
echoClient.SetAttribute("Interval", ns.core.TimeValue(ns.core.Seconds(1.0)))
echoClient.SetAttribute("PacketSize", ns.core.UintegerValue(1024))
```

We have to configure the client application and define what it will do. In this case, it tries to connect to the server by first fetching the address of the server. Then we are creating a client application by telling it to connect to the server address with port number 9. Then we define some attributes of the client application. We set the **MaxPacket** to fix how many packets the client will send the server. Then, we set the **Interval** to fix the delay between successive packet sending. Then, we set the **PacketSize** and tell the client application to send 1024 bytes of data to the server.

iii. Install the client application:

```
clientApps = echoClient.Install(nodes.Get(0))
clientApps.Start(ns.core.Seconds(2.0))
clientApps.Stop(ns.core.Seconds(10.0))
```

We now install the client application on the first node and set the start and stop time of the client application.

e. Running the simulation:

```
ns.core.Simulator.Run()
ns.core.Simulator.Destroy()
```

7. Observing the output:

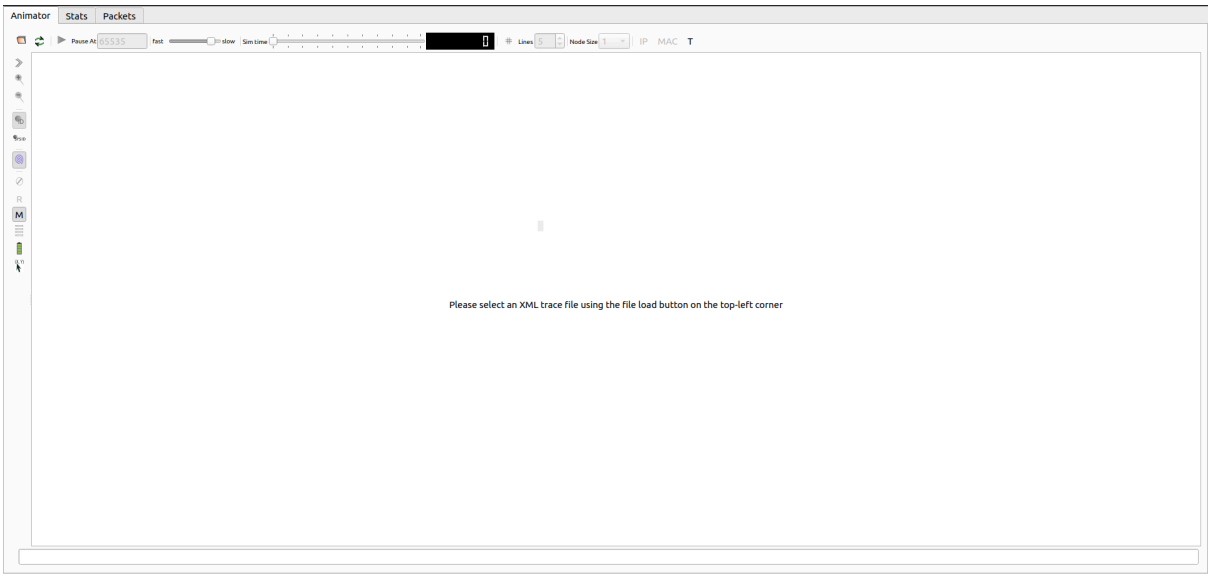
```
At time +2s client sent 1024 bytes to 10.1.1.2 port 9
At time +2.00369s server received 1024 bytes from 10.1.1.1 port 49153
At time +2.00369s server sent 1024 bytes to 10.1.1.1 port 49153
At time +2.00737s client received 1024 bytes from 10.1.1.2 port 9
```

Here we can see the traces of the network events that are performed by the server and client. We can see that 1024 bytes of data is being sent and received. Also, port 9 is used by the server as we had configured. We can also see the time in the simulation when the events happened. The transmission started at 2s because we configured the client to start at 2s.

8. Visualizing the simulation:
   a. Installing NetAnim:
   NetAnim is a simulation visualization tool that comes bundled with ns3. To use NetAnim, we need to perform some steps.
      i. Install prerequisite packages by running the following command:
         - sudo apt install qtbase5-dev qtchooser qt5-qmake qtbase5-dev-tools

      ii. Build NetAnim:
         - Move inside the **netanim-3.109** directory.
         - Run the following commands:
            a. qmake NetAnim.pro
            b. make

      iii. Run NetAnim:
         - **./NetAnim**
         - The interface should look like the following:



b. Generating the visualization script:
   i. Add the following line of code **before calling** the Run() method:
      - **anim = ns.netanim.AnimationInterface("first.xml")**
   ii. This will generate a “first.xml” file after running the simulation. This file contains the necessary scripts to run the visualization of our simulation.
c. Open the “first.xml” file in NetAnim and play the simulation. You will see a visual representation of the network events happened in our simulation.

9. Collecting network trace:
   We can collect packet captures from all our nodes in the simulation. These packet captures can help to understand the network events and troubleshoot the network.
   a. Add the following line of code **before calling** the Run() method:
      i. pointToPoint.EnablePcapAll("first")
   b. This will generate a pcap file for every node in our simulation. As we had only two nodes in our simulation, we will get two pacp files after running the simulation.
   c. Open the pcap files and observe the packets.
      i. For this we have to istall wireshark. So give command
         - **sudo apt install wireshark**
      ii. Then double click on the pcap files.

10. Getting network metrics and flow statistics:
    a. Add the following code block instead of calling the Run() method:

```
flowmon_helper = ns.flow_monitor.FlowMonitorHelper()
monitor = flowmon_helper.InstallAll()
monitor = flowmon_helper.GetMonitor()

ns.core.Simulator.Stop(ns.core.Seconds(20.0))
ns.core.Simulator.Run()

def print_stats(st):
    print("Tx Bytes: ", st.txBytes)
    print("Rx Bytes: ", st.rxBytes)
    print("Tx Packets: ", st.txPackets)
    print("Rx Packets: ", st.rxPackets)
    print("Lost Packets: ", st.lostPackets)
    if st.rxPackets > 0:
        print("Mean Delay: ", (st.delaySum.GetSeconds() / st.rxPackets))
        print("Throughput: ", (st.rxBytes*8)/18)

monitor.CheckForLostPackets()
classifier = flowmon_helper.GetClassifier()

for flow_id, flow_stats in monitor.GetFlowStats():
    t = classifier.FindFlow(flow_id)
    proto = {6: 'TCP', 17: 'UDP'} [t.protocol]
    print ("FlowID: %i (%s %s/%s --> %s/%i)" % \
        (flow_id, proto, t.sourceAddress, t.sourcePort, t.destinationAddress,
t.destinationPort))
    print_stats(flow_stats)
```

    b. Run the simulation and observe the output.

Home Task:

Now **vary the packet size and observe the metrics**. Take packet size [128,256,512,1024,2048] bytes and collect the throughputs for each of them. Plot the packet size vs Throughput in this case and explain the graph.